Deep Learning(CAI-466)

Objectives:

- Origin of Deep Learning
- Parameter Selection in Neural Networks
- Universal Function Approximation
- Width vs Depth
- Issues with depth
- Regularization

Origin of Deep Learning

Year	Contributer	Contribution
300 BC	Aristotle	introduced Associationism, started the history of human's attempt to understand brain.
1873	Alexander Bain	introduced Neural Groupings as the earliest models of neural network, inspired Hebbian Learning Rule.
1943	McCulloch & Pitts	introduced MCP Model, which is considered as the ancestor of Artificial Neural Model.
1949	Donald Hebb	considered as the father of neural networks, introduced Hebbian Learning Rule, which lays the foundation of modern neural network.
1958	Frank Rosenblatt	introduced the first perceptron, which highly resembles modern perceptron.
1974	Paul Werbos	introduced Backpropagation
1980	Teuvo Kohonen	introduced Self Organizing Map
	Kunihiko Fukushima	introduced Neocogitron, which inspired Convolutional Neural Network
1982	John Hopfield	introduced Hopfield Network
1985	Hilton & Sejnowski	introduced Boltzmann Machine
1986	Paul Smolensky	introduced Harmonium, which is later known as Restricted Boltzmann Machine
	Michael I. Jordan	defined and introduced Recurrent Neural Network
1990	Yann LeCun	introduced LeNet, showed the possibility of deep neural networks in practice
1997	Schuster & Paliwal	introduced Bidirectional Recurrent Neural Network
	Hochreiter & Schmidhuber	introduced LSTM, solved the problem of vanishing gradient in recurrent neural networks

2006	Geoffrey Hinton	introduced Deep Belief Networks, also introduced layer-wise pretraining technique, opened current deep learning era.
2009	Salakhutdinov & Hinton	introduced Deep Boltzmann Machines
2012	Geoffrey Hinton	introduced Dropout, an efficient way of training neural networks
2013	Kingma & Welling	introduced Variational Autoencoder (VAE), which may bridge the field of deep learning and the field of Bayesian probabilistic graphic models.
2014	Ian J. Goodfellow	introduced Generative Adversarial Network.
2015	Ioffe & Szegedy	introduced Batch Normalization

Parameter Selection in Neural Networks

•A deep neural network has large number of parameters
– Number of Layers

– Number of Neurons in Each Layer

–Activation Function for each neuron: ReLU, softmax, sigmoid, Tanh…

–Layer Connectivity: Dense, Dropout…

–Objective function
- •Loss Function: MSE, Entropy, Hinge loss, …
- •Regularization: L1, L2…

–Optimization Method
- •SGD, ADAM, RMSProp, LM …

–Parameters for the Optimization method
- •Weight initialization
- •Momentum, weight decay, etc.

Universal Function Approximation

•A neural network with one hidden layer can be used to approximate any shape. However, the approximation might require exponentially many neurons
–How can we reduce the number of computations? (PROBLEM)

Practical Issues in Universal Approximation

•The universal approximation theorem means that regardless of what function we are trying to learn, we know that a large neural network will be able to represent this function.

•However, we are not guaranteed that the training algorithm will be able to “learn” that function.
–Optimization can fail

–Learning is different from optimization (the primary requirement for learning is generalization)

Width vs. Depth

•An MLP with a single hidden layer is sufficient to represent any function

–But the layer may be infeasibly large

–May fail to learn and generalize correctly

•Using a deeper model can reduce the number of units required to represent the desired function and can reduce the amount of generalization error

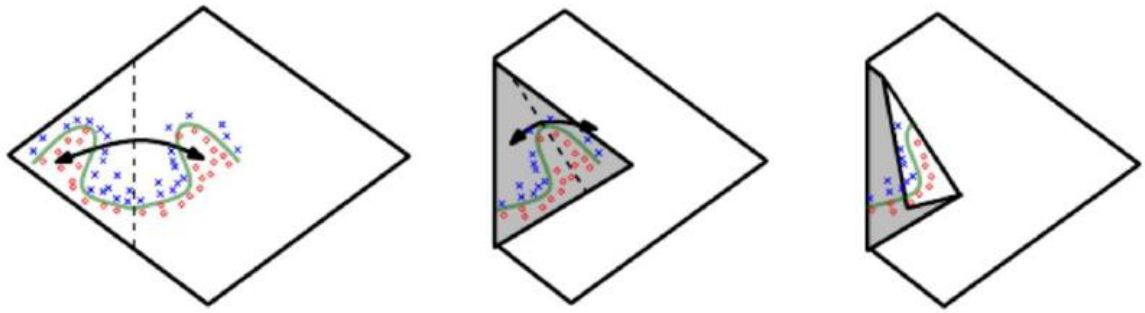
•A function that could be expressed with O(n) neurons on a network of depth k required at least $O(2\sqrt{n})$ and O((n -1)k) neurons on a two-layer neural network: Delalleau and Bengio(2011)

•Functions representable with a deep rectifier net can require an exponential number of hidden units with a shallow (one hidden layer) network: Montufar(2014)

•For a shallow network, the representation power can only grow polynomiallywith respect to the number of neurons, but for deep architecture, the representation can grow exponentially with respect to the number of neurons: Bianchini and Scarselli(2014)

•Depth of a neural network is exponentially more valuable than the width of a neural network, for a standard MLP with any popular activation functions: Eldanand Shamir (2015)

Exponential advantage of deeper networks



Empirical results for some data showed that depth increases generalization performance in a variety of applications.

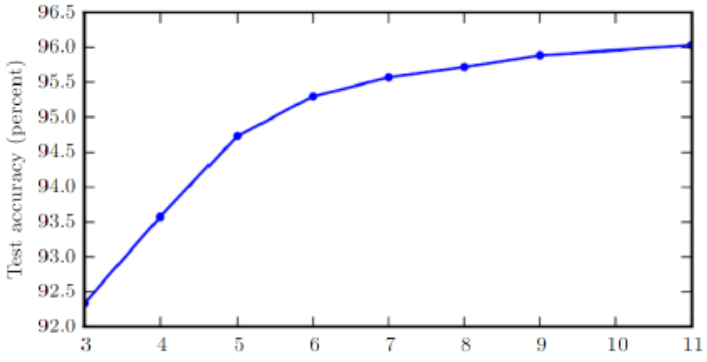


Figure 6.6: Empirical results showing that deeper networks generalize better when used to transcribe multi-digit numbers from photographs of addresses. Data from Goodfellow *et al.* (2014d). The test set accuracy consistently increases with increasing depth. See figure 6.7 for a control experiment demonstrating that other increases to the model size do not yield the same effect.

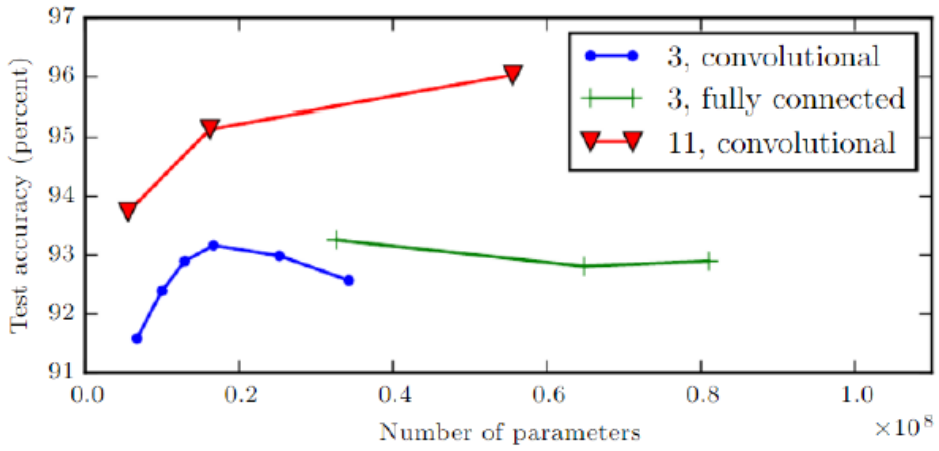
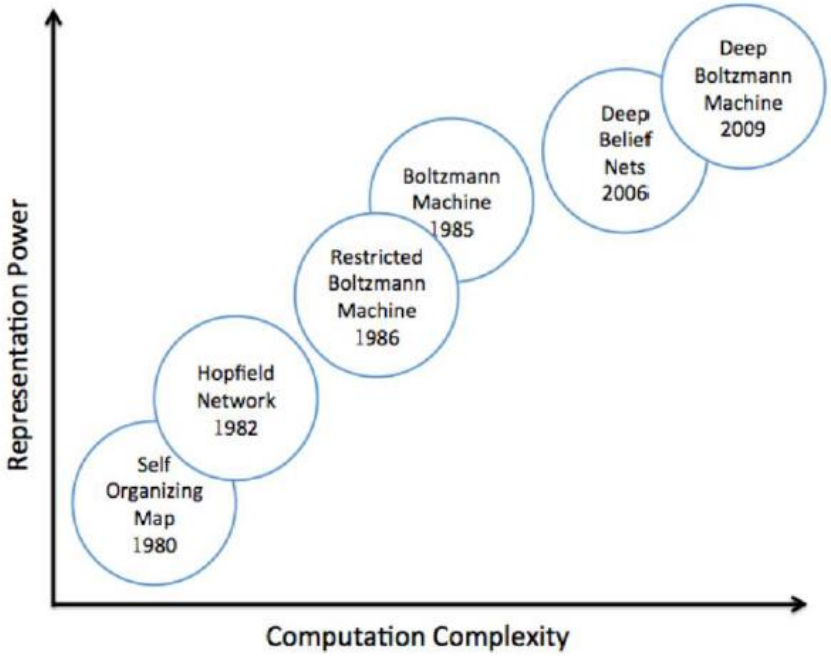


Figure 6.7: Deeper models tend to perform better. This is not merely because the model is larger. This experiment from Goodfellow *et al.* (2014d) shows that increasing the number of parameters in layers of convolutional networks without increasing their depth is not nearly as effective at increasing test set performance. The legend indicates the depth of network used to make each curve and whether the curve represents variation in the size of the convolutional or the fully connected layers. We observe that shallow models in this context overfit at around 20 million parameters while deep ones can benefit from having over 60 million. This suggests that using a deep model expresses a useful preference over the space of functions the model can learn. Specifically, it expresses a belief that the function should consist of many simpler functions composed together. This could result either in learning a representation that is composed in turn of simpler representations (e.g., corners defined in terms of edges) or in learning a program with sequentially dependent steps (e.g., first locate a set of objects, then segment them from each other, then recognize them).

Increasing representation power with depth



Issues with Depth

•Generalization

–Large networks are large capacity machines

–Remember: Learning requires generalization and goes beyond mere minimization of an objective function!

•Failure to Optimize

–Random initialization leads to the network being stuck in poor solutions

–Deeper networks are more prone to vanishing/exploding gradients and optimization failures

•"Greedy Layer-Wise Training of Deep Networks" by Bengioet al., 2006

–Uses unsupervised pre-training to initialize the weights of a network such that the optimization becomes easier

•Since 2010, this has been replaced with Drop-out and batch-normalization schemes which improve the optimization performance

–Rectified Linear Units get rid of the vanishing gradient problem

–Drop-out improves generalization

–Batch Normalization accelerates deep learning and improves generalization

»Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift by Ioffeand Segedy, 2015.

•Large scale optimization is tricky in deep learning

–Computationally demanding

–Requires efficient methods

•Stochastic gradient and sub-gradient methods

•Handling variety of neural network architectures

–How can we develop a framework of learning in which we can add layers, have a large diversity of layer connectivity, change objective functions and losses, layer connectivity, regularization, etc.?

–And still solve the optimization problem in an efficient manner!

–Symbolic Computation and Automatic Differentiation

–GPU

•Efficient matrix operations

•Higher bandwidth

Making deeper neural networks practical

•Optimization

•Handling vanishing (or exploding) gradients

–Pre-training (chose initial weights wisely)

–Drop-out

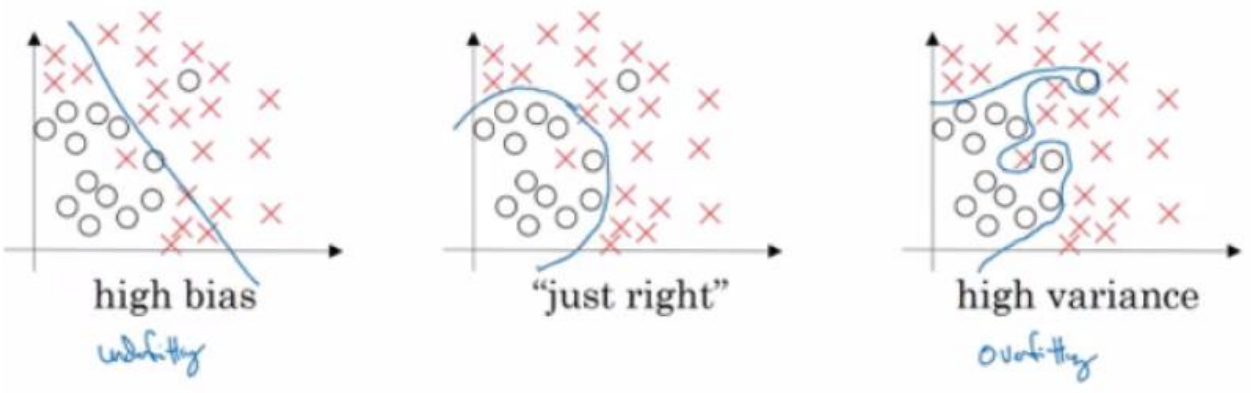
–Batch Normalization

•Computational challenges

–Use of computational graphs for automatic differentiation of neural networks
•Allows for different types of architectures
–Using GPU parallelization

Bias and Variance

Bias / Variance techniques are Easy to learn, but difficult to master, so here is the explanation of Bias and Variance:
If your model is underfitting (logistic regression of non linear data) it has a "high bias". If your model is overfitting then it has a "high variance". Your model will be alright if you balance the Bias / Variance.



Another idea to get the bias / variance if you don't have a 2D plotting mechanism:
High variance (overfitting) for example:
Training error: 1%
Dev error: 11%

High Bias (underfitting) for example:
Training error: 15%
Dev error: 14%

High Bias (underfitting) && High variance (overfitting) for example:
Training error: 15%
Test error: 30%

Best:
Training error: 0.5%
Test error: 1%

If your algorithm has a high bias:

- Try to make your NN bigger (size of hidden units, number of layers)
- Try a different model that is suitable for your data.
- Try to run it longer.
- Different (advanced) optimization algorithms.

If your algorithm has a high variance:

- More data.

- Try **regularization**.
- Try a different model that is suitable for your data.

You should try the previous two points until you have a low bias and low variance.
In the older days before deep learning, there was a "Bias/variance tradeoff". But because now you have more options/tools for solving the bias and variance problem its really helpful to use deep learning. Training a bigger neural network never hurts.

Regularization in Neural Networks

We have several means by which to help "regularize" neural networks –that is, to prevent overfitting

- Regularization penalty in cost function
- Dropout
- Early stopping
- Stochastic / Mini-batch Gradient descent (to some degree)

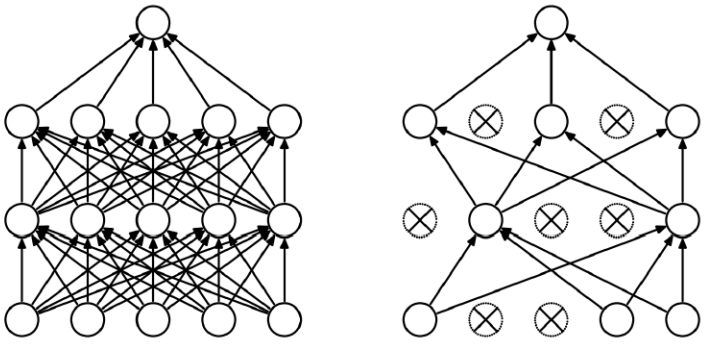
1- Penalized Cost function

- One option is to explicitly add a penalty to the loss function for having high weights.
- This is a similar approach to Ridge Regression
- Can have an analogous expression for Categorical Cross Entropy

$$J = \frac{1}{2n}\sum_{i=1}^{n}(\hat{y}_i - y_i)^2 + \lambda \sum_{j=1}^{m} W_i^2$$

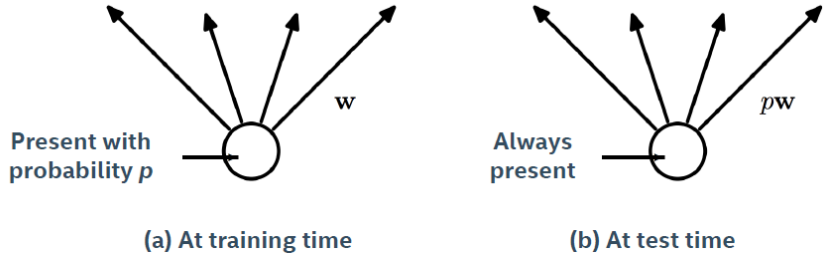
2- Dropout

Dropout is a mechanism where at each training iteration (batch) we randomly remove a subset of neurons. This prevents the neural network from relying too much on individual pathways, making it more "robust". At test time we "rescale" the weight of the neuron to reflect the percentage of the time it was active.



(a) Standard Neural Net (b) After applying dropout

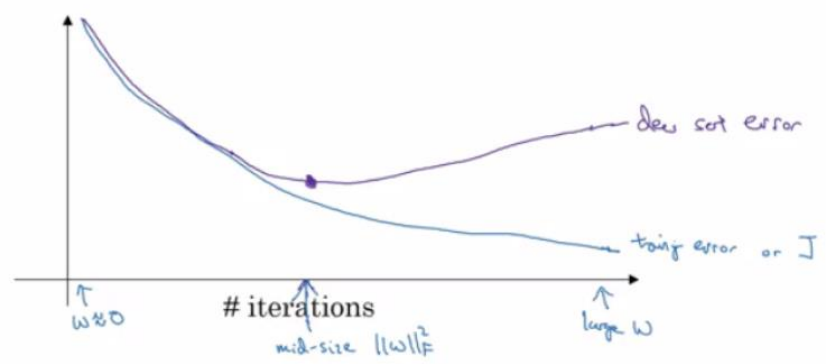
If the neuron was present with probability *p*, at test time we scale the outbound weights by a factor of *p*.



(a) At training time (b) At test time

3- Early Stopping

In this technique we plot the training set and the test set cost together for each iteration. At some iteration the test set cost will stop decreasing and will start increasing. We will pick the point at which the training set error and test set error are best (lowest training cost with lowest dev cost). We will take these parameters as the best parameters.



4- Optimizers

We have considered approaches to gradient descent which vary the number of data points involved in a step.

However, they have all used the standard update formula:

$$W := W - \alpha \cdot \nabla J$$

There are several variants to updating the weights which give better performance in practice.

These successive "tweaks" each attempt to improve on the previous idea.

The resulting (often complicated) methods are referred to as "**optimizers**".

5- Normalizing Inputs

If you normalize your inputs this will speed up the training process a lot. Normalization is done using these steps:

i. Get the mean of the training set: mean = (1/m) * sum(x(i))

ii. Subtract the mean from each input: X = X - mean

iii. Get the variance of the training set: variance = (1/m) * sum(x(i)^2)

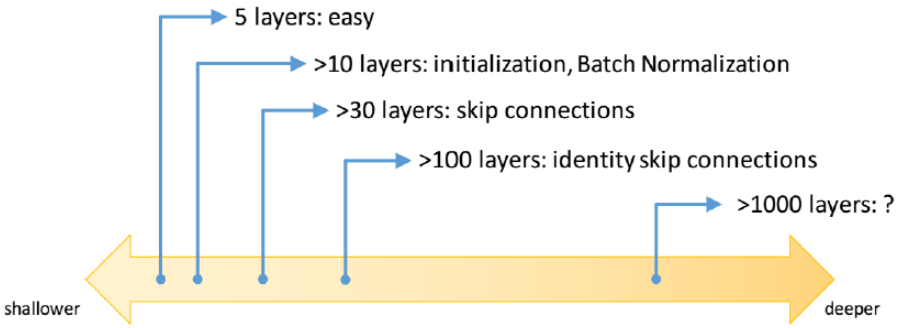
iv. Normalize the variance. X /= variance

These steps should be applied to training and testing sets (but using mean and variance of the train set).

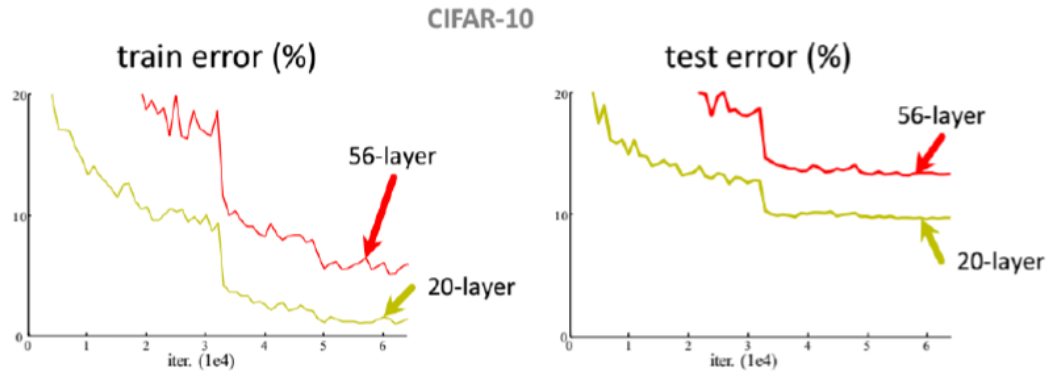
If we don't normalize the inputs our cost function will be deep and its shape will be inconsistent (elongated) then optimizing it will take a long time. But if we normalize it the opposite will occur. The shape of the cost function will be consistent (look more symmetric

like circle in 2D example) and we can use a larger learning rate alpha - the optimization will be faster.

Spectrum of Depth



What if we keep on stacking layers?
–56-layer net has **higher training error** and test error than 20-layer net



Kaiming He, Xiangyu Zhang, Shaoqing Ren, & Jian Sun. "Deep Residual Learning for Image Recognition". CVPR 2016

"Overly deep" plain nets have **higher training error**
•A general phenomenon, observed in many datasets
•Reasons
–Optimization failure